

# TERM\_5

## Local Cognitive Runtime Architecture

Start-to-Finish System Design for a Parallel, Reasoning-Aware, FIM-Enabled Terminal Agent

### Architecture position

Term\_5 is a fully local application. The user machine owns identity, state, memory, workspace intelligence, scheduling, tools, transactions, audit data, sessions, UI state, and all persistence. The DeepSeek API is used only as a remote inference provider. No ai.syntal.pro dependency, Syntal cloud dependency, or vision pipeline is required for v5.0.

|                 |                                            |
|-----------------|--------------------------------------------|
| Document status | Architecture baseline                      |
| Target release  | term_5 v5.0                                |
| Baseline        | term_4 behavior and capabilities           |
| Primary model   | DeepSeek Flash, per supplied model profile |
| Vision          | Explicitly out of scope for v5.0           |

*Design objective: Retrieve without reasoning. Complete with FIM. Reason where uncertainty exists. Parallelize independent work. Verify every action. Persist only what matters.*

# Contents

1. Executive Summary
2. Scope, Goals, and Non-Goals
3. Architecture Principles
4. DeepSeek Flash Model Profile and Compute Modes
5. Cognitive Model: Brain Analogy
6. Top-Level System Architecture
7. Runtime and Process Model
8. Source Package Architecture
9. End-to-End Turn Lifecycle
10. Attention and Reasoning Router
11. FIM Editing Pipeline
12. Parallel Scheduler and Task DAG
13. Global Workspace and Result Synthesis
14. Context Compiler and Prompt Cache Stability
15. Local Memory Architecture
16. Workspace Graph and Repository Intelligence
17. Tool System and Capability Model
18. Security Architecture
19. Transactional Editing and Parallel Writes
20. DeepSeek Provider Adapter
21. Local Persistence and Data Layout
22. Console and Local Web UI
23. Session Lifecycle and Recovery
24. Observability, Cost, and Performance
25. Doctor, Invariants, and Self-Verification
26. Testing and Evaluation Strategy
27. Failure Handling and Degradation
28. Privacy and Threat Model
29. Migration from term\_4 to term\_5
30. Implementation Plan
31. Acceptance Criteria
32. Reference Interfaces and Schemas
33. Example Execution Flows
34. Configuration Baseline
35. Final Architecture Summary

# 1. Executive Summary

term\_5 is the next-generation local cognitive runtime for the existing terminal agent line. It preserves the proven behavior of term\_4 while replacing the single-file architecture with explicit subsystems for cognition, memory, workspace intelligence, tools, security, scheduling, transactions, provider integration, interfaces, and verification.

The central architectural change is that DeepSeek is not treated as the agent itself. DeepSeek Flash is an inference engine used by a persistent local Brain. The Brain remains on the user machine and owns the durable state that makes the system coherent across turns and sessions.

The runtime has four compute paths: deterministic local operations, non-thinking model calls, adaptive reasoning calls, and FIM completion for localized code generation. An attention router selects the cheapest adequate path per operation. Complex work is decomposed into a dependency graph and independent nodes can execute concurrently under bounded resource budgets.

- Local-first: state, memory, workspace index, task graph, audit trail, transactions, and UI remain on the user machine.
- Adaptive reasoning: no thinking for deterministic work; low, high, or max reasoning only when justified.
- FIM as a first-class editing mechanism for bounded code insertion/replacement after intent is established.
- Parallel execution through dependency-aware task DAGs rather than unrestricted multi-agent spawning.
- One persistent Brain and many temporary workers; workers are scoped cognitive processes, not independent identities.
- Cache-stable context inherited from term\_4 and formalized as a Context Compiler.
- Capability-based security and transactional writes, with critical invariants enforced independently of the model.
- Vision is intentionally excluded from v5.0.

## Core definition

term\_5 = Local Brain + Context Compiler + Memory + Workspace Graph + Executive Scheduler + Tool Runtime + Transaction Manager + Security Policy + DeepSeek Provider + Console/Web UI.

# 2. Scope, Goals, and Non-Goals

## 2.1 Goals

- Preserve all valuable term\_4 behavior while making the codebase modular and testable.
- Use DeepSeek Flash efficiently across non-thinking, reasoning, tool-calling, JSON, and FIM workloads.
- Exploit concurrency for independent operations without multiplying unnecessary reasoning cost.
- Make every write recoverable and every high-impact decision verifiable.
- Provide the same Brain through the terminal and a loopback-only local web UI.
- Maintain durable local memory without making stale repository facts silently authoritative.
- Make performance, context use, cache effectiveness, costs, worker activity, and failures observable.

## 2.2 Non-goals for v5.0

- No vision or image understanding path.
- No dependency on ai.syntal.pro or any Syntal-hosted control plane.
- No multi-user cloud service.
- No autonomous always-on background daemon as a core requirement.
- No attempt to maximize worker count simply because provider concurrency is available.
- No direct model access to secrets, unrestricted shell execution, or unbounded write authority.
- No rewrite of term\_4 behavior without regression coverage.

# 3. Architecture Principles

| Principle       | Meaning                                                               |
|-----------------|-----------------------------------------------------------------------|
| Local ownership | All durable cognition and project state belongs to the local runtime. |

|                          |                                                                                                                |
|--------------------------|----------------------------------------------------------------------------------------------------------------|
| Model as compute         | DeepSeek performs inference; it does not own memory, identity, permissions, or application state.              |
| Reason selectively       | Use reasoning only when uncertainty, ambiguity, conflict, or planning complexity requires it.                  |
| Parallelize independence | Run work concurrently only when nodes are logically independent or safely isolated.                            |
| Verify actions           | A generated change is provisional until deterministic and/or reasoning-based verification passes.              |
| Cache stable context     | Keep stable prompt prefixes byte-stable and append volatile context in controlled layers.                      |
| Least capability         | Workers receive only the capabilities necessary for their task.                                                |
| One writer principle     | Default to a single effective writer per overlapping scope; isolate parallel writes in transactions.           |
| State is explicit        | No hidden global mutable state across subsystems; state ownership is defined.                                  |
| Failure is visible       | Security, memory, index, provider, and verification failures must not masquerade as empty or successful state. |

## 4. DeepSeek Flash Model Profile and Compute Modes

The architecture assumes the DeepSeek Flash profile supplied for the project. These values are treated as deployment inputs rather than hard-coded truths, so the provider adapter exposes them through configuration and runtime capability discovery.

| Capability             | v5 use                                                           |
|------------------------|------------------------------------------------------------------|
| Model                  | DeepSeek-V4.1-Flash / deepseek-flash                             |
| Context length         | 1M tokens                                                        |
| Maximum output         | 384K tokens                                                      |
| Thinking               | Supported; selectively enabled                                   |
| Tool calls             | Supported                                                        |
| JSON output            | Supported                                                        |
| Responses API          | Supported                                                        |
| FIM                    | Supported in non-thinking mode                                   |
| Vision                 | Supported by provider but disabled in term_5 v5.0                |
| Advertised concurrency | 2500; treated as an upper service ceiling, not a per-turn target |

### 4.1 Compute modes

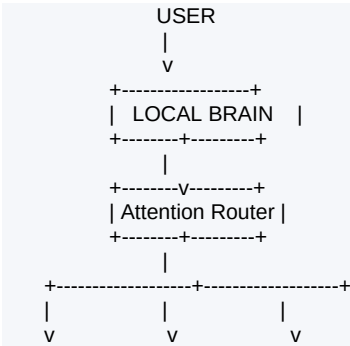
| Mode          | Provider behavior | Typical use                                                                                                |
|---------------|-------------------|------------------------------------------------------------------------------------------------------------|
| Deterministic | No model          | File reads, search, index lookup, syntax checks, hashing, git status, tests when command is predetermined. |
| Reflex        | Non-thinking chat | Classification, simple formatting, narrow interpretation, low-cost transformations.                        |

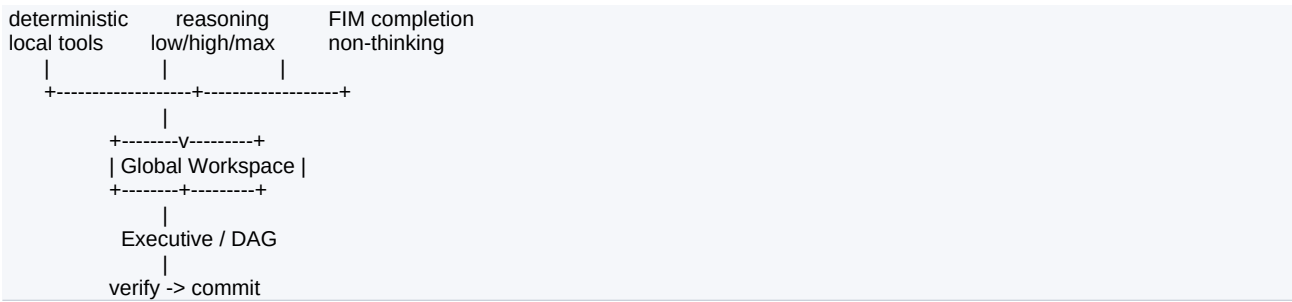
|            |                         |                                                                                                      |
|------------|-------------------------|------------------------------------------------------------------------------------------------------|
| Focus      | Low reasoning           | Small code decisions, lightweight conflict resolution, modest semantic analysis.                     |
| Deliberate | High reasoning          | Debugging, planning, architecture, dependency analysis, non-trivial code review.                     |
| Deep       | Max reasoning           | Conflicting evidence, high-risk design decisions, difficult synthesis, security-sensitive reasoning. |
| FIM        | Non-thinking completion | Bounded code insertion/replacement using prefix + suffix context.                                    |

## 5. Cognitive Model: Brain Analogy

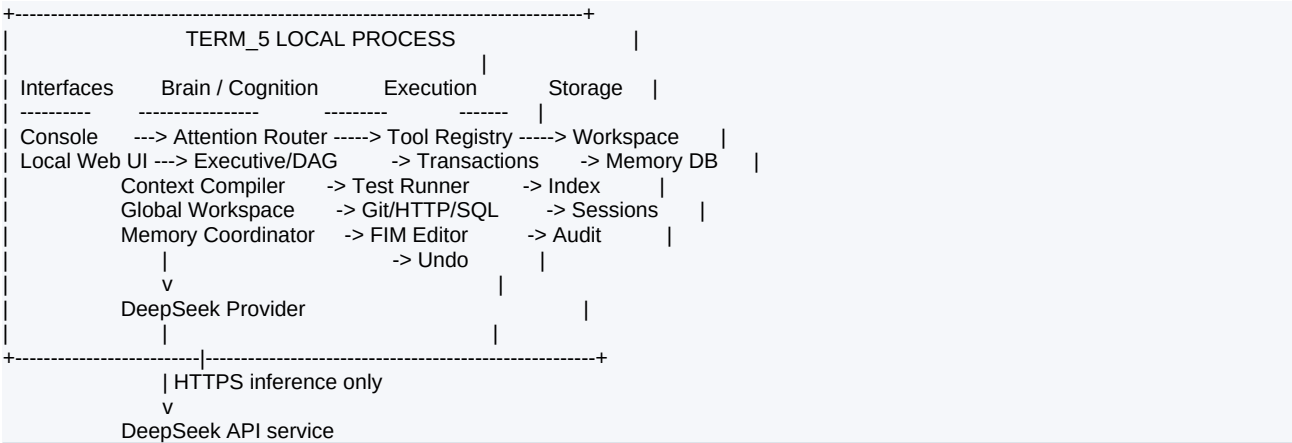
The human-brain comparison is used only as an engineering analogy. term\_5 is not modeled as biological neural tissue. The analogy helps separate fast automatic processing, attention, working memory, durable memory, executive control, action, and error monitoring.

| Brain analogy        | term_5 subsystem    | Role                                                                                                    |
|----------------------|---------------------|---------------------------------------------------------------------------------------------------------|
| Attention            | Attention Router    | Selects whether a task needs tools, FIM, low/high/max reasoning, or parallel decomposition.             |
| Executive control    | Executive Scheduler | Maintains the goal, decomposes work, controls budgets, orders dependencies, and stops unnecessary work. |
| Working memory       | Global Workspace    | Holds current goal, active hypotheses, task results, decisions, and current context.                    |
| Episodic memory      | Episode Store       | Records what occurred in prior turns and work sessions.                                                 |
| Semantic memory      | Fact Store          | Stores durable project/user facts with source, confidence, and freshness.                               |
| Procedural memory    | Skills              | Reusable procedures, scripts, and task-specific instructions.                                           |
| Automatic completion | FIM Engine          | Completes bounded code regions after higher-level intent is known.                                      |
| Motor execution      | Tool Runtime        | Performs file, process, git, HTTP, database, and archive operations.                                    |
| Error monitoring     | Verifier + Doctor   | Runs checks, tests invariants, detects unsafe or inconsistent state.                                    |
| Learning from error  | Lessons             | Captures durable constraints learned from failed verification or explicit correction.                   |





## 6. Top-Level System Architecture



All components above the external API boundary are local. DeepSeek receives only the context intentionally compiled for the current inference request. The local runtime remains authoritative for permissions, memory, task state, source files, transactions, and final commit decisions.

## 7. Runtime and Process Model

### 7.1 One primary process

The default deployment is one Python process per interactive term\_5 instance. Within that process, asynchronous tasks and bounded thread/subprocess pools provide parallelism. The architecture avoids requiring a local microservice mesh for v5.0.

### 7.2 Core runtime object

```
runtime = AgentRuntime(
    config=config,
    provider=DeepSeekProvider(config),
    context=ContextCompiler(...),
    memory=MemorySystem(...),
    workspace=WorkspaceGraph(...),
    tools=ToolRegistry(...),
    scheduler=CognitiveScheduler(...),
    transactions=TransactionManager(...),
    security=SecurityPolicy(...),
    sessions=SessionStore(...),
    events=EventBus(...),
)
```

CLI, local web UI, --once mode, evaluation harness, and tests all exercise the same AgentRuntime instead of carrying separate logic.

### 7.3 Concurrency domains

| Domain | Mechanism | Rule |
|--------|-----------|------|
|--------|-----------|------|

|                    |                     |                                                                           |
|--------------------|---------------------|---------------------------------------------------------------------------|
| Inference workers  | async I/O           | DeepSeek requests; bounded semaphores by reasoning class.                 |
| Local reads/search | async/thread pool   | Independent file/index reads and metadata operations.                     |
| Process/test jobs  | subprocess pool     | Tests, Python execution, allowed commands; bounded by CPU and policy.     |
| Transactions       | isolated workspaces | Parallel edit candidates; never uncontrolled writes to shared live files. |
| UI streaming       | event subscribers   | Console/web receive events without owning engine state.                   |

## 8. Source Package Architecture

```

term5/
|-- __main__.py
|-- cli.py
|-- config.py
|-- runtime.py
|
|-- brain/
|   |-- attention.py
|   |-- executive.py
|   |-- scheduler.py
|   |-- task_graph.py
|   |-- worker.py
|   |-- synthesis.py
|   `-- global_workspace.py
|
|-- agent/
|   |-- turn.py
|   |-- context.py
|   |-- prompts.py
|   |-- messages.py
|   |-- compaction.py
|   `-- events.py
|
|-- providers/
|   |-- base.py
|   |-- deepseek.py
|   `-- openai_compat.py
|
|-- editing/
|   |-- planner.py
|   |-- locator.py
|   |-- fim.py
|   |-- patch.py
|   |-- transaction.py
|   `-- validator.py
|
|-- tools/
|   |-- registry.py
|   |-- base.py
|   |-- files.py
|   |-- search.py
|   |-- git.py
|   |-- python.py
|   |-- tests.py
|   |-- http.py
|   |-- sqlite.py
|   |-- archive.py
|   `-- memory.py
|
|-- workspace/
|   |-- paths.py
|   |-- index.py
|   |-- graph.py
|   |-- brief.py
|   `-- watcher.py

```

```

|-- memory/
|-- store.py
|-- recall.py
|-- episodes.py
|-- lessons.py
|-- staleness.py
|-- reconcile.py

|-- state/
|-- session.py
|-- plan.py
|-- pins.py
|-- undo.py
|-- audit.py

|-- security/
|-- paths.py
|-- secrets.py
|-- network.py
|-- capabilities.py
|-- confirmation.py

|-- ui/
|-- console.py
|-- web/
|   |-- server.py
|   |-- api.py
|   |-- static/

|-- evals/
|-- doctor.py
|-- invariants.py
|-- scenarios.py
|-- regression/

```

The split is intentionally dependency-oriented. Security and storage primitives sit below cognition. The Brain may depend on security interfaces, but security code must never depend on model reasoning.

## 9. End-to-End Turn Lifecycle

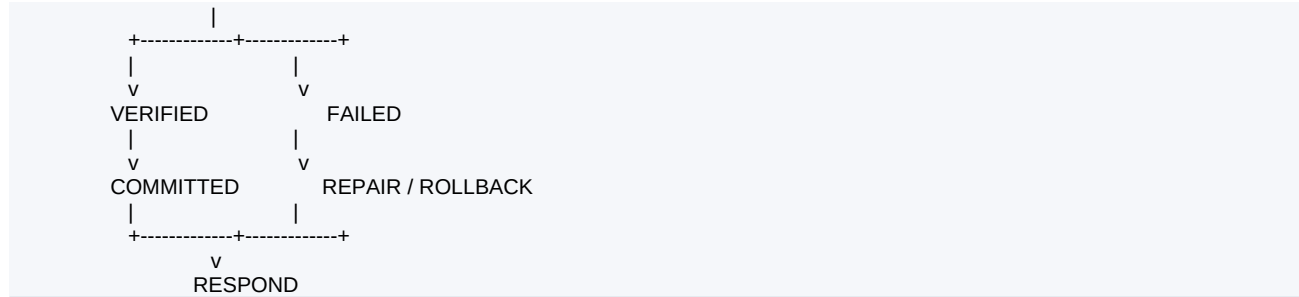
1. Accept the user input and create a Turn object with a unique local turn ID.
2. Classify the request locally where possible; otherwise use a minimal non-thinking classification call.
3. Compile the current goal, relevant memory, workspace signals, active plan, pins, and capability constraints.
4. Let the Attention Router choose deterministic, reflex, reasoning, FIM, or decomposed execution.
5. For complex tasks, the Executive builds a TaskGraph with explicit dependencies and risk levels.
6. Scheduler launches all ready read-only/independent nodes within budget.
7. Workers publish structured ResultEnvelopes into the Global Workspace; private reasoning is not blindly propagated.
8. Executive evaluates evidence, resolves conflicts, and creates execution/edit intents.
9. Edits run in transactions; FIM is preferred for bounded local code generation when appropriate.
10. Deterministic checks and targeted tests execute before transaction commit.
11. High-risk or ambiguous changes receive reasoning-based review after deterministic checks.
12. Commit accepted transactions, update workspace graph, invalidate stale facts, and write audit entries.
13. Consolidate only durable facts/lessons into long-term memory.
14. Synthesize the user-visible answer and stream it through the active interface.
15. Persist the session checkpoint and cost/usage metrics.

### 9.1 Turn state machine

```

RECEIVED
|
v
CLASSIFIED --> SIMPLE --> EXECUTE --> RESPOND
|
v
DECOMPOSED --> RUNNING_PARALLEL --> SYNTHESIS
|
v
PROVISIONAL

```



## 10. Attention and Reasoning Router

Reasoning is a scheduling decision, not a session-wide flag. Each task node declares the minimum cognitive mode needed. The router may upgrade a node when evidence conflicts or downgrade when deterministic evidence resolves the problem.

### 10.1 Routing signals

- Can the answer be produced from deterministic local state?
- Is the request merely classification/transformation or does it require inference?
- Is there ambiguity about user intent, architecture, causality, or correctness?
- Does the operation affect security, data loss, public interfaces, schemas, authentication, or multiple dependent modules?
- Are there independent subproblems that benefit from parallel evidence gathering?
- Can a bounded FIM completion implement a known edit without re-solving the design?
- Did prior verification fail in a way that requires conceptual reasoning rather than local repair?

### 10.2 Default policy

```

if deterministic(task):
    mode = LOCAL
elif localized_code_completion(task) and edit_intent_is_known(task):
    mode = FIM
elif trivial_semantic(task):
    mode = REFLEX
elif modest_uncertainty(task):
    mode = FOCUS
elif complex_or_cross_module(task):
    mode = DELIBERATE
else:
    mode = DEEP

# escalation
if evidence_conflicts or security_risk_high or repeated_failure:
    mode = upgrade(mode)
  
```

### 10.3 Reasoning budgets

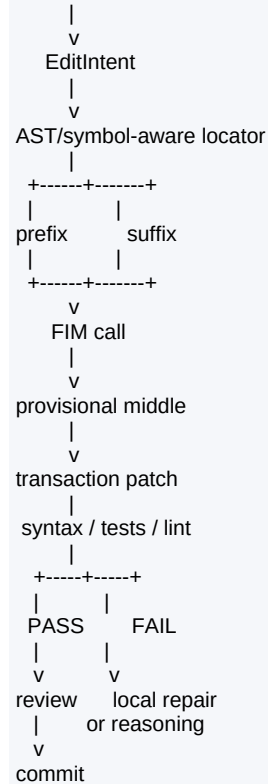
| Class      | Reasoning | Typical parallel workers | Notes                                                   |
|------------|-----------|--------------------------|---------------------------------------------------------|
| REFLEX     | none      | 1                        | Simple classifications or narrow transformations.       |
| FOCUS      | low       | 1-4                      | Small semantic/code reasoning.                          |
| DELIBERATE | high      | 2-12                     | Debugging, plans, architecture, reviews.                |
| DEEP       | max       | 1-4                      | Conflict resolution or high-risk synthesis.             |
| FIM        | none      | 1-64 typical             | Independent bounded code completions after plan exists. |

## 11. FIM Editing Pipeline

FIM is treated as a code execution/generation primitive rather than a general planner. It runs only after the Brain knows the target file, target symbol or region, intended change, and constraints. For v5.0, FIM always runs in non-thinking mode.

### 11.1 FIM flow

Reasoning / deterministic analysis



### 11.2 FIM request object

```

FimRequest(
    file="src/session.py",
    symbol="SessionManager.refresh",
    prefix=...,
    suffix=...,
    objective="Reject expired sessions before token rotation",
    constraints=[
        "Preserve the public method signature",
        "Do not change non-expired behavior",
    ],
    max_output_tokens=1024,
)

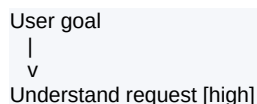
```

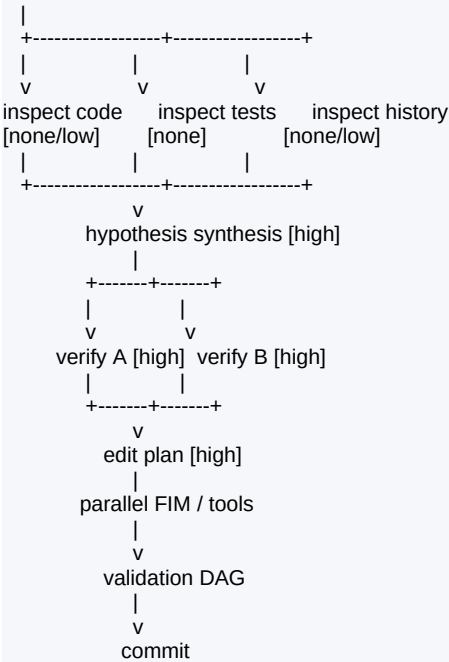
### 11.3 Parallel FIM

Independent files may receive FIM operations concurrently. Each operation writes into an isolated transaction. The merge engine rejects overlapping modifications until they are reconciled. FIM worker count is bounded by local merge complexity and validation capacity, not merely API concurrency.

## 12. Parallel Scheduler and Task DAG

All non-trivial parallel work is represented as a directed acyclic graph. A node runs when all dependencies are satisfied and its required capabilities and budgets are available.





12.1 Worker budgets

| Pool                          | Default                | Rationale                                           |
|-------------------------------|------------------------|-----------------------------------------------------|
| Inference total               | 32 default             | Global per runtime; configurable.                   |
| High/max reasoning            | 8 default              | Prevents expensive reasoning fan-out.               |
| FIM                           | 32 default             | May increase for bulk independent migrations.       |
| Read-only local operations    | 64 default             | Bounded by file descriptors/CPU.                    |
| Process/test jobs             | CPU-aware, usually 2-8 | Avoid saturating machine.                           |
| Effective overlapping writers | 1                      | Parallel writes only through isolated transactions. |
| Network/HTTP tools            | 8 default              | Separate from model concurrency.                    |

12.2 Scheduling policies

- Priority: user-blocking verification > execution > evidence gathering > optional enrichment.
- Cancellation propagates downstream when a parent hypothesis is disproved or the user cancels.
- Identical read-only tasks are deduplicated by task fingerprint.
- Workers have deadlines and token budgets.
- Backpressure is applied when local CPU, memory, test pool, or provider error rates exceed thresholds.
- The scheduler stops spawning additional workers when marginal evidence gain is low.

13. Global Workspace and Result Synthesis

Temporary workers do not share their full internal histories. They publish compact structured results into a local Global Workspace. This keeps context bounded and prevents one worker from polluting every other worker with irrelevant reasoning.

```
ResultEnvelope(  
  task_id="inspect_db_transaction",  
  status="success",  
  conclusion="Rollback path can leave the session cache stale",  
  evidence=[  
    Evidence(file="session_store.py", lines=[188, 214]),
```

```
Evidence(test="test_refresh_rollback", result="fails"),
],
artifacts=[...],
confidence="high",
followups=["inspect cache invalidation"],
)
```

13.1 Synthesis rules

- Prefer evidence-backed conclusions over worker self-confidence.
- Deduplicate equivalent findings.
- Preserve disagreements as explicit conflicts until resolved.
- Escalate to deliberate/deep reasoning only when the conflict matters to the goal.
- Do not copy full worker contexts into the final synthesis unless required for traceability.
- Persist only user-relevant conclusions, durable facts, or lessons after the turn completes.

14. Context Compiler and Prompt Cache Stability

term\_4 introduced the key optimization: stable system material remains byte-identical while volatile context is appended per turn. v5 formalizes this into a Context Compiler with explicit layers, priorities, token limits, and cache policies.

| Layer | Content                     | Lifetime                    | Priority | Cache behavior             |
|-------|-----------------------------|-----------------------------|----------|----------------------------|
| 0     | Agent instructions          | session/static              | Highest  | Stable                     |
| 1     | Workspace brief             | session-frozen              | High     | Stable                     |
| 2     | Loaded skills               | session or explicit refresh | High     | Stable until changed       |
| 3     | Conversation history        | append-only                 | High     | Append-only cache-friendly |
| 4     | Relevant memory             | per turn                    | High     | Volatile                   |
| 5     | Plan / pins / project delta | per turn                    | High     | Volatile                   |
| 6     | Worker evidence             | per task/synthesis          | Medium   | Volatile                   |
| 7     | Current user input          | current turn                | Highest  | Newest tail                |

14.1 Context budget example

| Budget component                | Illustrative cap |
|---------------------------------|------------------|
| Stable system + skills          | 30K              |
| Workspace context               | 50K              |
| Relevant memory/lessons         | 30K              |
| Conversation                    | 600K             |
| Worker evidence/tool results    | 150K             |
| Reserved response/tool headroom | 140K             |

Caps are policy limits, not guaranteed allocations. The compiler uses priority-aware pruning, summaries, result compaction, and deduplication. A 1M provider context is not treated as permission to inject every available byte.

14.2 Context observability

```
/context
total request      284,112 / 1,000,000
stable prefix      24,880
cache-eligible history  173,442
current memory     8,120
tool evidence      51,670
```

|                 |         |
|-----------------|---------|
| current turn    | 26,000  |
| reserved        | 715,888 |
| prompt rebuilds | 0       |

## 15. Local Memory Architecture

Memory is local and stratified. It is not a transcript dump. Each memory type has a different retention policy and trust model.

| Memory type | Store                 | Contains                                                  | Retention                        |
|-------------|-----------------------|-----------------------------------------------------------|----------------------------------|
| Working     | Turn/global workspace | Current goal, active hypotheses, in-flight task state     | Ephemeral                        |
| Episodic    | Episode store         | What happened in prior interactions or project operations | Bounded/compacted                |
| Semantic    | Fact store            | Durable facts about project/user/workflow                 | Long-lived with source/freshness |
| Lessons     | Lesson store          | Constraints learned from failures/corrections             | Long-lived until superseded      |
| Procedural  | Skills                | Reusable operating procedures and scripts                 | Versioned files                  |

### 15.1 Semantic fact schema

```
{
  "id": "mem_0193",
  "type": "fact",
  "scope": "project",
  "text": "Authentication sessions are persisted in PostgreSQL",
  "source": {"kind": "file", "path": "config/database.py"},
  "confidence": 0.97,
  "created_at": "...",
  "last_verified": "...",
  "freshness": "current",
  "tags": ["auth", "database"]
}
```

### 15.2 Staleness and invalidation

Repository-derived memories are linked to source fingerprints. When a source file or relevant symbol changes, affected facts become possibly\_stale. Retrieval can still surface them, but the Brain must verify stale facts before using them as authoritative evidence.

```
source fingerprint changed
|
v
related facts -> possibly_stale
|
+--> irrelevant to current task: leave stale
|
--> needed now: verify -> refresh / supersede / delete
```

## 16. Workspace Graph and Repository Intelligence

The term\_4 knowledge index evolves into a Workspace Graph. The graph is incremental, local, and designed to answer impact questions before edits occur.

- File nodes: path, language, size, hash, modified time, parse status.
- Symbol nodes: classes, functions, methods, constants, exported objects.
- Edges: defines, imports, imported\_by, calls where detectable, tested\_by, configured\_by, sibling, generated\_from.
- Project metadata: git branch, dirty state, package manifests, test framework, build markers.
- Freshness: dirty nodes are reparsed after writes or filesystem changes.

### 16.1 Impact-aware editing

```
auth/session.py
|
+--> imported_by --> api/login.py
+--> imported_by --> api/register.py
+--> imported_by --> middleware/session.py
|
|--> tested_by ----> tests/test_auth.py
                    tests/test_session.py
```

Edit planner receives this neighborhood before proposing a patch.

## 17. Tool System and Capability Model

Tools are declarative objects registered through a ToolRegistry. The registry centralizes validation, permissions, audit logging, timing, output limits, secret redaction, confirmation, undo snapshots, and event emission.

```
@tool(
    name="write_file",
    risk="write",
    capabilities={"workspace.write"},
)
async def write_file(ctx, path: str, content: str):
    ...
```

### 17.1 Capability vocabulary

| Domain    | Capabilities                                                      |
|-----------|-------------------------------------------------------------------|
| Workspace | workspace.read, workspace.write, workspace.delete, workspace.move |
| Process   | process.tests, process.python, process.command                    |
| Network   | network.fetch, network.request, network.secret_send               |
| Database  | database.read, database.write                                     |
| Git       | git.read, git.commit                                              |
| Memory    | memory.read, memory.write, memory.forget                          |
| Archive   | archive.read, archive.write                                       |

### 17.2 Worker capability profiles

| Worker             | Capabilities                       | Restriction                                              |
|--------------------|------------------------------------|----------------------------------------------------------|
| Repository scout   | workspace.read                     | Cannot modify files.                                     |
| Test scout         | workspace.read, process.tests      | May run bounded tests; no writes.                        |
| FIM editor         | workspace.read + transaction.write | Writes only to assigned isolated transaction.            |
| Verifier           | workspace.read, process.tests      | Reads candidate transaction and runs checks.             |
| Network researcher | network.fetch                      | No secrets unless separately granted.                    |
| Commit worker      | git.read, git.commit               | Only after approved transaction and confirmation policy. |

## 18. Security Architecture

Security is implemented below the cognition layer. A model instruction can request an operation, but it cannot bypass the SecurityPolicy. Critical guards are deterministic and independently tested.

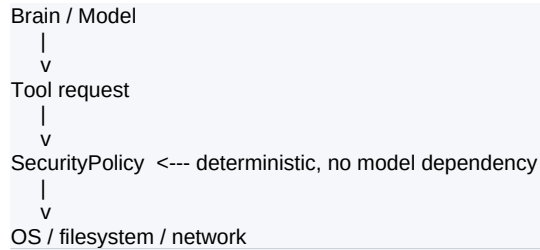
## 18.1 Path safety

- All workspace paths are canonicalized before authorization.
- Paths outside the configured workspace root are rejected.
- Symlink traversal is resolved before authorization.
- Protected patterns (.env, .git internals, SSH keys, credential locations, caches as configured) are denied.
- Nonexistent paths are authorized against their canonical parent chain before creation.
- Path guard tests cover ../ traversal, absolute escapes, symlink chains, protected files, Unicode/case behavior where platform-relevant, and nested valid paths.

## 18.2 Network and secrets

- Private, loopback, link-local, and metadata destinations are denied for generic remote fetches unless explicitly part of a safe local interface path.
- Secret substitution happens after policy validation and is never emitted into model-visible logs.
- Requests containing secret references require an explicit host allowlist.
- Redirect chains are revalidated.
- Download sizes and timeouts are bounded.

## 18.3 Security dependency rule

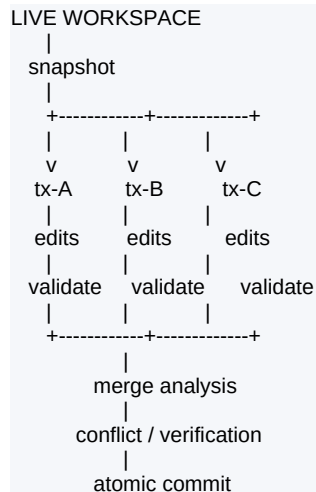


### Fail-closed rule

If a critical security invariant fails, write/delete/network-secret/process-write capabilities are disabled until the invariant is restored. A failed doctor must not be treated as an informational warning for critical guards.

# 19. Transactional Editing and Parallel Writes

Every multi-step edit runs inside a transaction. The live workspace is not the scratchpad. This is essential once FIM and multiple workers operate concurrently.



## 19.1 Transaction properties

- Each transaction has an ID, base workspace fingerprint, changed-file set, diff, validation report, and provenance.
- Parallel transactions may automatically merge only when changes are non-overlapping or safely mergeable.

- Overlapping symbol/line changes require reconciliation.
- Commit is atomic at the transaction level as far as practical; pre-write backups support rollback.
- After commit, the Workspace Graph is updated and dependent memory facts are invalidated.
- Failed transactions remain inspectable but never silently alter the live workspace.

## 20. DeepSeek Provider Adapter

Provider-specific behavior is isolated behind a `ModelProvider` interface. `term_5` cognition should not import DeepSeek-specific response types outside the provider package.

```
class ModelProvider(Protocol):
    async def complete(self, request: ChatRequest) -> ChatResponse: ...
    async def fim(self, request: FimRequest) -> FimResponse: ...
    async def health(self) -> ProviderHealth: ...
    def capabilities(self) -> ProviderCapabilities: ...
```

### 20.1 Request classes

| Request class     | Use                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Chat non-thinking | Simple semantic tasks, formatting, classification.                                                     |
| Reasoning low     | Focus mode.                                                                                            |
| Reasoning high    | Deliberate mode, most complex coding/debugging.                                                        |
| Reasoning max     | Conflict/high-risk synthesis.                                                                          |
| FIM non-thinking  | Prefix/suffix completion for bounded code changes.                                                     |
| Tool loop         | Reasoning or non-thinking chat with tool call continuation; provider state fields preserved correctly. |

### 20.2 Provider safeguards

- Central retry and exponential backoff with jitter.
- Per-mode concurrency semaphores.
- Rate/error feedback into scheduler backpressure.
- Request IDs recorded locally for diagnostics without recording secrets.
- Explicit timeouts and cancellation.
- No automatic fallback to more expensive/deeper reasoning without scheduler approval.
- Capability discovery and configuration validation at startup.

## 21. Local Persistence and Data Layout

All application state is persisted locally beneath a `term_5`-owned state directory. Project-local cache data may remain inside the workspace when useful, while user-global secrets and history remain under the user configuration/home directories.

```
<workspace>/
.term5_cache/
  workspace.sqlite
  graph/
  transactions/
  undo/
  audit.jsonl
  runtime.log

~/.config/term_5/
  config.toml
  secrets.json
  profiles/

~/.local/share/term_5/
  sessions/
  memory/
  facts.sqlite
```

```
episodes.jsonl
lessons.jsonl
```

```
~/term5_history
```

## 21.1 Storage choices

SQLite is preferred for structured indexes, graph metadata, and semantic memory because it provides transactions and queryability without a separate local service. JSONL remains appropriate for append-only logs and episodes. Human-readable mirrors may be generated, but the structured store remains authoritative.

## 22. Console and Local Web UI

Both interfaces are thin clients over the same AgentRuntime and EventBus. The GUI must not have a second execution engine.

### 22.1 Console

- Streaming assistant text and reasoning status (without exposing private chain-of-thought).
- Tool cards/status lines.
- Slash commands for status, context, memory, plan, doctor, tasks, transactions, cost, and sessions.
- Interactive confirmation prompts for destructive/high-risk actions.

### 22.2 Local web UI

- Binds only to loopback by default.
- Token-gated local session.
- Host/Origin protections and restrictive CSP.
- Streams events from EventBus.
- Panels for memory, plan, task DAG, active workers, transactions, workspace graph, cost/context, sessions, doctor, and secrets management.
- Confirmation modal maps to the same ConfirmationService as console.

### 22.3 UI event model

```
TurnStarted
TaskGraphCreated
WorkerStarted
ModelRequestStarted
ToolRequested
ToolStarted
ToolFinished
TransactionChanged
VerificationFinished
WorkerFinished
SynthesisStarted
TurnFinished
ConfirmationRequested
ErrorRaised
```

## 23. Session Lifecycle and Recovery

16. Start runtime and acquire workspace identity/fingerprint.
17. Load configuration and validate provider credentials/capabilities.
18. Run critical startup invariants.
19. Load or create local session.
20. Freeze cache-stable workspace brief and system prefix.
21. Load memory/index metadata and reconcile stores.
22. Process turns; append checkpoints after completed phases.
23. On clean exit, flush session, graph, metrics, and logs.
24. On crash/restart, inspect incomplete transactions and in-flight session checkpoint before resuming.

### 23.1 Crash recovery

An incomplete transaction is never auto-committed after restart. The runtime marks it abandoned or recoverable and requires revalidation before any merge. Session messages may resume from the last committed checkpoint; in-flight worker results are treated as disposable unless durably checkpointed.

## 24. Observability, Cost, and Performance

term\_5 must make its cognition measurable. Metrics are local and should be useful without external telemetry.

```
/status
TERM_5 5.0.0

SESSION
  turns      38
  uptime     2h14m

COGNITION
  active DAG nodes    4
  running workers     6
  high/max workers    2
  FIM workers         3

CONTEXT
  current request     284K / 1M
  stable prefix       24.9K
  prompt rebuilds     0

MODEL
  calls              71
  cache hit tokens    1.82M
  cache miss tokens   311K
  output tokens       143K
  estimated spend     $...

WORKSPACE
  files              412
  symbols            3,814
  dirty graph nodes   0

SECURITY
  doctor            PASS
```

### 24.1 Performance objectives

- Simple deterministic commands should avoid model latency entirely.
- Read-only evidence gathering should execute concurrently when independent.
- Reasoning requests should be fewer and more targeted than retrieval/tool operations.
- FIM should minimize regenerated code and validation scope.
- Cache-stable prefixes should remain unchanged through normal turns.
- Local scheduler backpressure should protect the workstation even when provider concurrency is high.

## 25. Doctor, Invariants, and Self-Verification

The doctor is not a cosmetic health page. It proves critical runtime properties in-process without requiring a model call wherever possible.

| Invariant                | Severity | Expected proof                                             |
|--------------------------|----------|------------------------------------------------------------|
| Path confinement         | CRITICAL | Escapes and unsafe symlinks are blocked.                   |
| Protected path blocklist | CRITICAL | Secrets/internal caches are inaccessible to general tools. |
| Secret redaction         | CRITICAL | Secret values never appear in tool/model-visible traces.   |
| Secret host gate         | CRITICAL | Secret-bearing network calls require allowed hosts.        |

|                      |          |                                                             |
|----------------------|----------|-------------------------------------------------------------|
| Memory store/mirror  | HIGH     | Divergence is detected and reconciled.                      |
| Workspace graph      | HIGH     | Index DB readable and schema current.                       |
| Transaction rollback | CRITICAL | Failed edit restores prior state.                           |
| Prompt stability     | HIGH     | Stable prefix remains byte-identical across ordinary turns. |
| Tool registry        | HIGH     | Schemas, handlers, and capabilities agree.                  |
| Provider health      | MEDIUM   | Remote inference reachable; graceful offline state if not.  |
| FIM path             | MEDIUM   | Completion adapter and validator operational.               |
| Event bus            | MEDIUM   | Subscribers cannot crash runtime.                           |

**Critical startup behavior**

If a critical invariant fails, term\_5 enters a restricted mode that disables the affected high-risk capabilities. It must not silently continue as fully operational.

## 26. Testing and Evaluation Strategy

### 26.1 Unit tests

- Security/path/network guards.
- Context layer ordering and token budgeting.
- Memory staleness/invalidation.
- Workspace graph parsing and update behavior.
- Tool registry validation.
- Transaction isolation, merge, rollback.
- Task DAG dependency resolution and cancellation.
- Provider response normalization.
- FIM boundary construction and patch validation.

### 26.2 Integration tests

- Full turn with deterministic tools only.
- Reasoning turn with tool continuation.
- Parallel read-only task graph.
- Parallel FIM edits on independent files.
- Conflicting transaction merge.
- Test failure -> FIM repair -> revalidation.
- Crash between transaction write and commit.
- Memory fact invalidated by source change.
- Console and GUI both driving the same runtime.

### 26.3 Regression evaluation

term\_4 is the behavioral reference during migration. Existing term\_4 offline scenarios and live task scenarios should be ported before subsystem replacement. A v5 migration step is accepted only when its regression score is at least equivalent for the covered behavior.

### 26.4 Benchmark classes

| Benchmark      | Measure                          |
|----------------|----------------------------------|
| Simple command | Latency and zero-model-call rate |

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| Repository question | Correct evidence selection and token use            |
| Bug fix             | Tests passed, diff quality, calls, reasoning depth  |
| Bulk migration      | Parallel speedup, merge conflicts, FIM success rate |
| Long session        | Cache stability, context compaction, memory quality |
| Safety              | Escape/secret/destructive refusal correctness       |

## 27. Failure Handling and Degradation

| Failure                    | Behavior                                                                                                    |
|----------------------------|-------------------------------------------------------------------------------------------------------------|
| DeepSeek unavailable       | Continue deterministic local tools; report model path unavailable; do not corrupt session.                  |
| Rate limiting              | Scheduler reduces concurrency and retries within policy.                                                    |
| FIM failure                | Retry once if transient; otherwise fall back to reasoning-guided patch or manual patch tool.                |
| Reasoning failure          | Use existing evidence; optionally retry at same/higher mode within budget.                                  |
| Test failure               | Transaction remains provisional; run repair path or rollback.                                               |
| Memory corruption          | Mark memory subsystem degraded; reconcile from mirrors/backups; never pretend memory is empty.              |
| Index corruption           | Rebuild workspace graph from files; keep writes restricted if impact analysis depends on unavailable graph. |
| Security invariant failure | Disable affected risky capabilities.                                                                        |
| Worker timeout             | Cancel node, preserve partial evidence only if valid, and let Executive decide fallback.                    |
| UI disconnect              | Runtime continues; user may reconnect locally; console may remain available.                                |

## 28. Privacy and Threat Model

The primary privacy advantage of v5 is local ownership of durable state. However, prompts sent to DeepSeek still leave the machine. The Context Compiler therefore becomes a privacy boundary as well as a token-management component.

### 28.1 Threats

- Prompt injection in repository files/web content.
- Path traversal or symlink escape.
- Secret exfiltration through tool calls or model context.
- Malicious URLs / SSRF / redirect abuse.
- Model-generated destructive commands.
- Concurrent transaction races.
- Stale memory causing unsafe assumptions.
- Over-broad context sending unnecessary proprietary code to the provider.
- Compromised local browser origin interacting with the web UI.

### 28.2 Controls

- Treat file/web content as untrusted data, not executable instruction.
- Deterministic path and network guards.
- Secret vault with token substitution after authorization.
- Capability-based worker permissions.

- Transaction isolation and confirmation policy.
- Source-linked memory freshness.
- Minimum necessary context compilation.
- Loopback UI, session token, Host/Origin checks, CSP.
- Audit trail for security-relevant operations.

## 29. Migration from term\_4 to term\_5

v5 should be built as a staged extraction, not a clean-room rewrite. term\_4 is the specification for preserved behavior; modules are replaced behind regression tests.

| Stage            | Work                                                                         | Exit gate                                   |
|------------------|------------------------------------------------------------------------------|---------------------------------------------|
| 0 — Freeze       | Capture term_4 selftests/evals and representative sessions.                  | Baseline score and known failures recorded. |
| 1 — Foundation   | Extract config, security, storage, logging, events.                          | Behavior unchanged; tests pass.             |
| 2 — Tool runtime | Move tools behind ToolRegistry + capabilities.                               | Tool schema/handler regression passes.      |
| 3 — AgentRuntime | Create runtime object and remove global ownership where possible.            | Console/once use same runtime.              |
| 4 — Context      | Replace prompt assembly with Context Compiler preserving v4 cache stability. | Prefix stability tests pass.                |
| 5 — Brain        | Add Attention Router, Executive, TaskGraph, Scheduler, Global Workspace.     | Parallel read-only scenarios pass.          |
| 6 — Editing      | Add transactions, locator, FIM, validators.                                  | Parallel edit/rollback scenarios pass.      |
| 7 — Memory/Graph | Add staleness and graph-backed impact analysis.                              | Freshness/regression scenarios pass.        |
| 8 — UI           | Move web UI fully onto EventBus/runtime.                                     | Console and GUI parity tests pass.          |
| 9 — Harden       | Doctor gating, performance, long-session tests.                              | Release acceptance criteria met.            |

## 30. Implementation Plan

### 30.1 Build order

25. Create package skeleton and typed configuration.
26. Port critical term\_4 tests before moving behavior.
27. Implement EventBus and structured logging.
28. Extract SecurityPolicy and fix/lock path safety invariants.
29. Extract persistence, sessions, memory stores, and workspace index adapters.
30. Create Tool base class, registry, capability checks, and port tools incrementally.
31. Create provider abstraction and DeepSeek adapter for chat/reasoning/tool loops.
32. Create AgentRuntime and make console/--once use it.
33. Implement Context Compiler and reproduce term\_4 cache-stability behavior.
34. Add TaskGraph, Attention Router, Scheduler, worker result envelopes, and Global Workspace.
35. Add TransactionManager and edit validation pipeline.
36. Add FIM adapter, semantic locator, repair loop, and parallel FIM support.
37. Evolve index into Workspace Graph and wire impact analysis.
38. Add memory source fingerprints and staleness.
39. Migrate local web UI to runtime events and task/transaction panels.
40. Complete doctor severity gating and long-session/performance evaluations.
41. Cut v5.0 only after regression and safety gates pass.

## 30.2 Suggested release slices

| Release    | Scope                                                              |
|------------|--------------------------------------------------------------------|
| 5.0-alpha1 | Modular runtime, config, security, ToolRegistry, provider adapter. |
| 5.0-alpha2 | Context Compiler, sessions, memory parity, console parity.         |
| 5.0-beta1  | Task DAG, bounded parallel workers, Global Workspace.              |
| 5.0-beta2  | Transactions + FIM + validation/repair.                            |
| 5.0-rc1    | Workspace Graph, staleness, GUI parity, observability.             |
| 5.0.0      | All acceptance criteria, doctor pass, regression parity or better. |

## 31. Acceptance Criteria

- **PASS:** All critical doctor invariants pass on supported platforms.
- **PASS:** No known regression in preserved term\_4 functionality within the covered eval suite.
- **PASS:** Normal multi-turn sessions keep the stable prompt prefix unchanged unless explicitly refreshed.
- **PASS:** Simple deterministic tasks can complete with zero model calls.
- **PASS:** Reasoning effort is observable per task and not globally forced.
- **PASS:** FIM is used only for bounded non-thinking code completion and its outputs are always provisional until validated.
- **PASS:** Independent task nodes demonstrably execute in parallel while respecting configured budgets.
- **PASS:** Parallel writes cannot corrupt the live workspace; conflicts are detected before commit.
- **PASS:** Failed edits rollback cleanly.
- **PASS:** Memory distinguishes empty, unavailable, stale, and no-match states.
- **PASS:** Workspace-derived facts become stale when their source fingerprints change.
- **PASS:** Console and local web UI use the same runtime and state.
- **PASS:** Vision dependencies are absent from the v5.0 execution path.
- **PASS:** The runtime functions without ai.syntal.pro or any Syntal-hosted service.
- **PASS:** Local state remains readable/recoverable without the DeepSeek API being available.

## 32. Reference Interfaces and Schemas

### 32.1 TaskNode

```
TaskNode(  
  id: str,  
  objective: str,  
  kind: LOCAL | CHAT | REASON | FIM | TOOL | VERIFY,  
  reasoning: NONE | LOW | HIGH | MAX,  
  dependencies: set[str],  
  capabilities: set[str],  
  risk: LOW | MEDIUM | HIGH | CRITICAL,  
  token_budget: int | None,  
  timeout_s: float | None,  
  transaction_id: str | None,  
)
```

### 32.2 WorkerResult

```
WorkerResult(  
  task_id: str,  
  status: SUCCESS | FAILED | CANCELLED | PARTIAL,  
  conclusion: str,  
  evidence: list[Evidence],  
  artifacts: list[ArtifactRef],  
  followups: list[str],  
  metrics: WorkerMetrics,  
)
```

### 32.3 EditIntent

```

EditIntent(
  file: str,
  target_symbol: str | None,
  objective: str,
  constraints: list[str],
  expected_tests: list[str],
  risk: RiskLevel,
)

```

## 32.4 ContextBlock

```

ContextBlock(
  id: str,
  content: str,
  priority: int,
  max_tokens: int,
  cache_policy: STATIC | SESSION | TURN | TASK,
  source: str,
  sensitivity: PUBLIC | PROJECT | SECRET_REF,
)

```

# 33. Example Execution Flows

## 33.1 “Show git status”

```

User -> Attention Router -> deterministic
  -> git.status tool
  -> format result
  -> User

```

Model calls: 0  
Reasoning: none

## 33.2 “Find why registration sometimes returns 500 and fix it”

```

Executive [HIGH]
|
+--> inspect registration code [NONE]
+--> inspect relevant tests [NONE]
+--> inspect error/log handling [LOW]
+--> inspect DB transaction path [HIGH]
+--> recall relevant lessons [NONE]
|
v
Hypothesis synthesis [HIGH]
|
+--> validate hypothesis A [HIGH]
`--> validate hypothesis B [HIGH]
|
v
EditIntent [HIGH]
|
v
FIM / transactional patch [NONE]
|
+--> syntax [LOCAL]
+--> targeted tests [LOCAL]
+--> dependency checks [LOCAL/LOW]
`--> diff review [HIGH]
|
v
commit -> update graph/memory -> answer

```

## 33.3 Bulk API rename across 40 independent files

```

One architecture reasoning pass [HIGH]
|
v
Generate migration rule
|
+--> 40 isolated FIM transactions [NONE, parallel]
|
v

```

```

parallel syntax/static validation
  |
  v
merge analysis
  |
  v
integration tests
  |
  v
reasoning review of failures only [HIGH]
  |
  v
atomic commit

```

## 34. Configuration Baseline

# term5.toml (illustrative)

```

[model]
provider = "deepseek"
model = "deepseek-flash"
context_limit = 1000000
max_output_tokens = 384000
vision = false

[reasoning]
default = "auto"
max_parallel_high = 8
max_parallel_total = 32

[fim]
enabled = true
max_parallel = 32
default_max_output_tokens = 2048

[scheduler]
max_local_reads = 64
max_process_jobs = 4
max_network_jobs = 8

[security]
confirm_destructive = true
fail_closed_on_critical_doctor = true

[context]
cache_stable = true
memory_mode = "index+relevant"

[ui]
web_enabled = true
web_host = "127.0.0.1"
open_browser = true

```

Configuration values should be validated against provider capabilities and local machine resources at startup. The runtime may reduce concurrency automatically but must not silently raise security permissions.

## 35. Final Architecture Summary

term\_5 should be built as a local cognitive operating layer over tools and a remote inference provider. Its identity is not the DeepSeek request loop. Its identity is the durable local Brain: context, memory, workspace knowledge, executive control, permissions, transactions, verification, and interface-independent state.

```

Retrieve without reasoning.
Complete bounded code with FIM.
Reason when uncertainty exists.
Parallelize only independent cognition.
Isolate every write.
Verify before commit.
Remember durable facts, not noise.
Keep state local.
Keep one Brain.

```

This architecture preserves term\_4’s strongest ideas — understanding-first behavior, persistent memory, self-verification, guarded tools, cache-stable prompts, cost awareness, and a local web UI — while giving v5 a structure that can scale to sophisticated parallel work without becoming another monolithic script.

## Appendix A — v5.0 Fixed Constraints

| Constraint         | Decision                                                     |
|--------------------|--------------------------------------------------------------|
| Deployment         | Local application on the user machine.                       |
| External inference | DeepSeek API only; provider abstraction retained.            |
| Primary model      | DeepSeek Flash profile supplied for the project.             |
| Vision             | Disabled / not implemented in v5.0.                          |
| Reasoning          | Adaptive none/low/high/max per task.                         |
| FIM                | Enabled, non-thinking, bounded completion path.              |
| Parallelism        | Dependency-aware, bounded pools, task DAG.                   |
| Persistence        | Local files/SQLite/JSONL; no remote control plane required.  |
| UI                 | Console + loopback local web UI over one runtime.            |
| Security           | Deterministic capability and policy layer beneath cognition. |